

Asymptotically Fast Algorithms for Modern Computer Algebra

Jürgen Gerhard
Maplesoft
Waterloo, Ontario, Canada
jgerhard@maplesoft.com

ABSTRACT

The solution of computational tasks from the “real world” requires high performance computations. Not limited to mathematical computing, asymptotically fast algorithms have become one of the major contributing factors in this area. Based on [4], the tutorial will give an introduction to the beauty and elegance of modern computer algebra.

Categories and Subject Descriptors

I.1.2 [Symbolic and Algebraic Manipulation]: Algorithms; F.2.1 [Analysis of Algorithms and Problem Complexity]: Numerical Algorithms and Problems—*computations on polynomials*; G.4 [Mathematics of Computing]: Mathematical Software—*algorithm design and analysis, efficiency*

General Terms

Theory

Keywords

fast algorithms, computer algebra

1. INTRODUCTION

Mathematical software has long advanced beyond its academic nursery and has become a standard tool in industrial design and analysis practice. The size of industrial problems made it necessary not just for numerical computation packages but also for computer algebra systems to increase their performance. More and more symbolic computation systems and packages, such as GMP [14], Magma [2], Maple [7], and NTL [10], now feature implementations of asymptotically fast algorithms that scale almost linearly with the input size.

2. FAST ALGORITHMS

At the heart of asymptotically fast polynomial arithmetic lies fast multiplication, e.g., by the fast Fourier transform [3].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ISSAC 2010, 25–28 July 2010, Munich, Germany.

Copyright 2010 ACM 978-1-4503-0150-3/10/0007 ...\$10.00.

An abundance of higher-level computational problems for polynomials, including but not limited to division [11], evaluation and interpolation [1], greatest common divisors [13], factorization [15], symbolic integration [5] and symbolic summation [6] can be reduced to polynomial multiplication, and any speedup in the underlying basic polynomial arithmetic immediately translates into a speedup of about the same order of magnitude for these advanced algorithms as well.

The techniques for fast symbolic computation have some of their roots in numerical analysis (fast Fourier transform, Newton iteration) and computer science (e.g., divide-and-conquer, work balancing). A somewhat unique ingredient in computer algebra, however, is the omnipresent and powerful scheme of modular algorithms.

The techniques mentioned above work well not only for polynomial arithmetic, but can be extended (with some notable exceptions) to integer arithmetic as well [9], and often the same or similar techniques apply or can be used in linear algebra [12].

Optimizations make asymptotically fast algorithms practical and powerful. Determining the break-even points between classical and fast algorithms for hybrid schemes can be challenging and platform dependent. The Golden Rules of Schönhage et al [8] for the development of fast algorithms apply, notably:

- Do care about the size of $O(1)$!
- Do not waste a factor of two!
- Don’t forget the algorithms in object design!
- Clean results by approximate methods is sometimes much faster!

3. REFERENCES

- [1] A. Borodin and I. Munro. *The Computational Complexity of Algebraic and Numeric Problems*, volume 1 of *Theory of computation series*. American Elsevier Publishing Company, New York, 1975.
- [2] J. J. Cannon and W. Bosma, editors. *Handbook of Magma Functions, Edition 2.13*. 2006. magma.maths.usyd.edu.au.
- [3] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19:297–301, 1965.
- [4] J. von zur Gathen and J. Gerhard. *Modern Computer Algebra*. Cambridge University Press, 2nd edition, 2003.

- [5] J. Gerhard. Fast modular algorithms for squarefree factorization and Hermite integration. *Applicable Algebra in Engineering, Communication and Computing*, 11(3):203–226, 2001.
- [6] J. Gerhard. *Modular Algorithms in Symbolic Summation and Symbolic Integration*, volume 3218 of *Lecture Notes in Computer Science*. Springer Verlag, 2004.
- [7] Maplesoft. Maple – Math & Engineering Software. www.maplesoft.com/products/Maple/index.aspx.
- [8] A. Schönhage, A. F. W. Grotefeld, and E. Vetter. *Fast Algorithms – A Multitape Turing Machine Implementation*. BI Wissenschaftsverlag, Mannheim, 1994.
- [9] A. Schönhage and V. Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, 7:281–292, 1971.
- [10] V. Shoup. NTL: A library for doing number theory. www.shoup.net/ntl.
- [11] M. Sieveking. An algorithm for division of powerseries. *Computing*, 10:153–156, 1972.
- [12] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13:354–356, 1969.
- [13] V. Strassen. The computational complexity of continued fractions. *SIAM Journal on Computing*, 12(1):1–27, 1983.
- [14] Various authors. The GNU multiple precision arithmetic library. www.gmp1ib.org.
- [15] H. Zassenhaus. On Hensel factorization, I. *Journal of Number Theory*, 1:291–311, 1969.